

Q1.

Data redundancy – The same customer address is stored in multiple spreadsheets.

Data inconsistency – When the customer updates the address only at the branch, the credit card department's copy remains outdated.

Q2

- i. Data integrity: Ensures rules such as unique account numbers and valid balances. Without it, it would lead to incorrect transactions or records.
- ii. Concurrency: multiple transactions at the same time. Two transactions updating the same account could cause an incorrect account balance.
- iii. Recovery: Recovers data after system crashes or failures. A database failure could lose customer account or transaction records.

Q4

- i. customer.customer_id: primary key
- ii. account.account_number: candidate key
- iii. account.customer_id: foreign key
- iv. account.balance: none
- v. employee.manager_id: foreign key
- vi. branch.ifsc_code: candidate

Q5

- i. A customer checks a balance on the ICICI mobile app.: OLTP
- ii. The risk team calculates non-performing loans by branch for the quarter.: OLAP
- iii. An ATM dispenses 10,000 rupees.: OLTP
- iv. A regulator asks for the total value of transactions above 200,000 in the last financial year.: OLAP
- v. A teller opens a new savings account.:OLTP
- vi. Marketing finds the top 100 customers by average monthly balance.: OLAP.

Q6

- i. **Performance impact:** Running them on the production database can consume CPU, memory, and I/O needed for real-time banking transactions.
- ii. **Concurrency and availability risk:** Heavy analytical queries running alongside OLTP transactions can increase same time uses, potentially affecting customer operations.
- iii. **Workload mismatch:** The core database is designed for **OLTP**—fast, small, frequent transactions—whereas a quarterly regulatory report is **OLAP** work involving large historical datasets and complex aggregations.

Use a separate reporting/data-warehouse environment populated from the core system

Q7

Atomicity -- d. Money is debited but never credited

Consistency – c. An account exists for a customer who does not

Isolation – b. Two ATMs both pay out against the same balance

Durability – a. A committed transfer disappears after a power cut

Q8

The **write-ahead log is written to disk before the actual data page** so that the database has a record of every change before that change effects the database.

Q9

`SELECT * FROM customer` makes the application depend on the table's current column structure. If the bank later adds a `pan_number` column, the query will automatically return that new column, which can break application code that expects the old order of columns, causing incorrect data mapping or display errors.

A safer approach is to explicitly request only the columns the application needs

Q10

- i. **Oracle:** Use it for core banking systems where strong transaction management, reliability, security, and enterprise-scale processing are required.
- ii. **PostgreSQL:** Use it for application databases where an open-source relational database with strong SQL support is suitable.
- iii. **MySQL:** Use it for web-based banking applications and supporting services where a widely adopted relational database is sufficient.
- iv. **Redis:** Use it for caching frequently accessed data and fast, low-latency operations such as sessions, temporary data, and frequently requested balances.

Q11

Write a query that returns the database name, the current user and the server port.

```
1 SELECT current_database(),
2     current_user,
3     current_setting('port');
```

	current_database name	current_user name	current_setting text
1	icicibank	postgres	5432

Q12

Using information_schema, list every column of the loan table with its data type and whether it accepts NULL. How many columns does loan have? What is the degree of the relation?

```
SELECT COUNT(*) AS number_of_columns
FROM information_schema.columns
WHERE table_name = 'loan';
```

	number_of_columns bigint
1	11

```
SELECT column_name, data_type, is_nullable
FROM information_schema.columns
WHERE table_name = 'loan'
ORDER BY ordinal_position;
```

	column_name name	data_type character varying	is_nullable character varying (3)
1	loan_id	integer	NO
2	customer_id	integer	NO
3	branch_id	integer	NO
4	loan_type	character varying	NO

Q13

Report the number of rows in all nine tables in a single result set, sorted by table name.

```
SELECT 'account' AS table_name, (SELECT COUNT(*) FROM account) AS row_count
UNION ALL
SELECT 'account_txn', (SELECT COUNT(*) FROM account_txn)
UNION ALL
SELECT 'beneficiary', (SELECT COUNT(*) FROM beneficiary)
UNION ALL
SELECT 'branch', (SELECT COUNT(*) FROM branch)
UNION ALL
SELECT 'card', (SELECT COUNT(*) FROM card)
```

Q14 List every foreign key in the database, showing the child table, the child column, and the parent table it points to. Which table has a foreign key pointing at itself, and what does that represent in the bank?

```
SELECT tc.table_name AS child_table, kcu.column_name AS child_column,
       ccu.table_name AS parent_table
FROM information_schema.table_constraints AS tc
JOIN information_schema.key_column_usage AS kcu
  ON tc.constraint_name = kcu.constraint_name
  AND tc.table_schema = kcu.table_schema
JOIN information_schema.constraint_column_usage AS ccu
  ON tc.constraint_name = ccu.constraint_name
  AND tc.table_schema = ccu.table_schema
WHERE tc.constraint_type = 'FOREIGN KEY'
ORDER BY tc.table_name, kcu.column_name;
|
```

Q15 Find the three largest tables on disk, including indexes, with sizes in a human-readable form. Why is `account_txn` the largest?

```
select relname AS table_name,
       pg_size_pretty(pg_total_relation_size(relid)) AS size
from pg_stat_user_tables
ORDER BY pg_total_relation_size(relid) DESC
LIMIT 3;|
```

Q16

ICICI Bank opens a current account for a new corporate customer. Insert the customer and the account using these details, then read both rows back.

```
INSERT INTO customer (
  customer_id,
  first_name,
  last_name,
  date_of_birth,
  email,
  phone,
  city,
  kyc_status,
  risk_rating,
  customer_since,
  home_branch_id
)
VALUES (
  1032,
  'Latha', 'Krishnan', '1985-09-23', 'latha.krishnan@example.com', '9845099887', 'Chennai', 'VERIFIED', 'MEDIUM', '2026-03-01', 5
);

INSERT INTO account (
  account_id,
  account_number,
  customer_id,
  branch_id,
  account_type,
  balance,
  opened_date,
  interest_rate
)
VALUES (
  9047,
  'ICIC0000009047', 1032, 5, 'CURRENT', 500000.00, '2026-03-01', 0.00
);

SELECT *
FROM customer
WHERE customer_id = 1032;

SELECT *
FROM account
WHERE account_id = 9047;
```

Q17

The customer deposits 75,000 and then withdraws 20,000. Write both updates so that they compute from the current balance rather than assigning an absolute figure, then confirm the final balance is 555,000.

```
update account
set balance=balance+75000
where account_id=9047;

select balance from account where account_id=9047;

update account
set balance=balance-20000
where account_id=9047;
```

Q18

Try to delete customer 1032 without deleting the account first. Record the exact error. Then delete both rows in the correct order and confirm the table counts are back to 30 and 45.

```
56 DELETE FROM customer
57 WHERE customer_id = 1032;
```

Data Output Messages Notifications

ERROR: update or delete on table "customer" violates foreign key constraint "fk_account_customer_id" on table "account"
Key (customer_id)=(1032) is still referenced from table "account".

SQL state: 23503
Detail: Key (customer_id)=(1032) is still referenced from table "account".

```
DELETE FROM account
WHERE account_id = 9047;

DELETE FROM customer
WHERE customer_id = 1032;

SELECT 'customer' AS table_name, COUNT(*) AS row_count
FROM customer

SELECT 'account', COUNT(*)
FROM account;
```

Q19 Write three statements that the database must reject, one for each of these reasons, and record the

error message for each:

1. An account whose account_type is FIXED.
2. A transaction whose amount is -5000.
3. A customer whose kyc_status is IN_PROGRESS.

```
21 INSERT INTO account (
22     account_id, account_number, customer_id, branch_id,
23     account_type, balance, opened_date
24 )
25 VALUES (
26     9991, 'ICIC0000009991', 1001, 5,
27     'FIXED', 1000.00, '2026-03-01'
28 );
```

Data Output Messages Notifications

ERROR: new row for relation "account" violates check constraint "ck_account_1"
Failing row contains (9991, ICIC0000009991, 1001, 5, FIXED, 1000.00, INR, ACTIVE, 2026-03-01, null).

SQL state: 23514
Detail: Failing row contains (9991, ICIC0000009991, 1001, 5, FIXED, 1000.00, INR, ACTIVE, 2026-03-01, null).

```

30 INSERT INTO account_txn (
31     txn_id, account_id, txn_date, txn_type,
32     amount, channel
33 )
34 VALUES (
35     9991, 9004, CURRENT_TIMESTAMP, 'DEBIT',
36     -5000, 'ATM'
37 );

```

Data Output Messages Notifications

ERROR: new row for relation "account_txn" violates check constraint "ck_account_txn_2"
Failing row contains (9991, 9004, 2026-09-17 14:13:26.168492, DEBIT, -5000.00, ATM, null, SUCCESS, null).

SQL state: 23514
Detail: Failing row contains (9991, 9004, 2026-09-17 14:13:26.168492, DEBIT, -5000.00, ATM, null, SUCCESS, null).

```

39 INSERT INTO customer (
40     customer_id, first_name, last_name, date_of_birth,
41     email, phone, city, kyc_status, customer_since, home_branch_id
42 )
43 VALUES (
44     9991, 'Test', 'Customer', '1990-01-01',
45     'test@example.com', '9999999999', 'Chennai',
46     'IN_PROGRESS', '2026-03-01', 5
47 );

```

Data Output Messages Notifications

ERROR: new row for relation "customer" violates check constraint "ck_customer_1"
Failing row contains (9991, Test, Customer, 1990-01-01, test@example.com, 9999999999, Chennai, IN_PROGRESS, null, 2026-03-01, 5).

SQL state: 23514
Detail: Failing row contains (9991, Test, Customer, 1990-01-01, test@example.com, 9999999999, Chennai, IN_PROGRESS, null, 2026-03-01, 5).

Q20 Using two sessions, demonstrate that an uncommitted UPDATE in Session A is invisible to Session B.

Show the SQL from both sessions and the output at each step.

```

50 SELECT account_id, balance
51 FROM account
52 WHERE account_id = 9002;
53
54 BEGIN;
55
56 UPDATE account
57 SET balance = balance + 10000
58 WHERE account_id = 9002;
59
60 SELECT account_id, balance
61 FROM account
62 WHERE account_id = 9002;

```

1	SELECT	account_id,	balance
2	FROM	account	
3	WHERE	account_id =	9002;

Data Output	Messages	Notifications						
≡+ 📄 ▼ 📋 ▼ 🗑️ 🗄️ ⬇️ 📈 5 <table> <tr> <th></th><th>account_id [PK] integer</th><th>balance numeric (15,2)</th></tr> <tr> <td>1</td><td>9002</td><td>43522.54</td></tr> </table>		account_id [PK] integer	balance numeric (15,2)	1	9002	43522.54		
	account_id [PK] integer	balance numeric (15,2)						
1	9002	43522.54						

Q21

In a single transaction, transfer 12,000 from account 9004 to account 9010, then roll it back. Prove with a query before and after that neither balance changed.

```
SELECT account_id, balance
FROM account
WHERE account_id IN (9004, 9010)
ORDER BY account_id;

BEGIN;
UPDATE account
SET balance = balance - 12000
WHERE account_id = 9004;

UPDATE account
SET balance = balance + 12000
WHERE account_id = 9010;
ROLLBACK;
```

```
SELECT account_id, balance
FROM account
WHERE account_id IN (9004, 9010)
ORDER BY account_id;
```

Q22 Show the current values of `shared_buffers`, `work_mem`, `wal_level` and `fsync`. For each, write one sentence explaining what it controls.

```
show shared_buffers;  
show work_mem;  
show wal_level;  
show fsync;
```

Q23

My recommendation would be not to use MySQL with the MyISAM storage engine for UPI transaction storage. MyISAM does not provide transaction support or row-level locking, creating a risk of inconsistent or lost transaction updates under high concurrent load such as 4,000 transactions per second.

The specific alternative is to use MySQL with the **InnoDB** storage engine, which provides transactions, ACID compliance, row-level locking, and better support for high-concurrency payment workloads.

Q24

Branch operations: needs fast **OLTP-style point queries** for one account, such as retrieving all transactions for a particular `account_id`, ideally supported by an index on (`account_id`, `txn_date`).

Risk team: needs **OLAP-style analytical queries** that scan and aggregate millions of `account_txn` rows by month and across all accounts; running these directly on the transactional database can consume CPU, memory, and I/O needed for daily banking operations.

Solution

Core/OLTP database — keep `account_txn` in the transactional system, optimized with indexes for branch screens and real-time transaction processing.

Separate OLAP/reporting database or data warehouse — continuously copy transaction data from the core system and optimize it for large monthly aggregations.

Q25

The statement “The application logged ‘transfer successful’, so the money must have moved” is not evidence because an application log is only a record of what the application tried to do. It could have logged success even if the database transaction was later reverted, failed, or never committed.

- i. A committed database transaction showing the debit and credit.
- ii. The before-and-after account balances confirming ₹12,000 was deducted from the sender and added to the receiver.
- iii. An outbox table record in account_txn with successful status.

Q26

Do not set fsync = off on the core banking PostgreSQL server. Although it may make batch loads faster, disabling fsync removes PostgreSQL's guarantee that committed data and WAL records are safely written to disk, creating a risk of **data loss or database corruption after a crash or power failure**.

If it must be used anywhere, restrict it to a **temporary, disposable development/test environment** or a rebuildable staging database where data loss is acceptable, never on production/core banking data.

Q27

- i. Protection from every source of data changes:
The database may be accessed by the backend services, admin tools, or other applications. A database constraint ensures that none of them can insert or update an account to a negative balance.
- ii. Centralized data integrity:
The rule is enforced consistently at the data-storage level. Even if an application has a bug or forgets to check the balance, the database rejects the invalid value. Your schema already enforces this with CHECK (balance >= 0).

A rule that depends on **complex business logic or external information** is often better handled in the application. For example, “*allow a withdrawal only if the customer's account has sufficient balance **and** the customer has not exceeded their daily withdrawal limit*” may require checking multiple transactions and business conditions, while the database CHECK constraint is suited to a simple row-level rule like balance >= 0.

Q28

SQL/database features: PostgreSQL and Oracle have different SQL syntax, functions, and database features, so existing queries and application code may need changes.

Transaction and concurrency behaviour: Their implementations of transactions, locking, and isolation can differ, potentially causing different behaviour under concurrent banking transactions.

Configuration and recovery: PostgreSQL settings such as `shared_buffers`, `work_mem`, `wal_level`, and `fsync` have PostgreSQL-specific meanings and configurations that do not directly map to Oracle.

Storage and performance design: PostgreSQL and Oracle use different storage engines and indexing approaches, so workloads optimized for one system may not perform the same way after migration.

Q29

Moving from spreadsheets to a relational database will give the bank one controlled place for customer and account information instead of maintaining separate copies across teams. This will reduce duplicate information, prevent different departments from holding different versions of the same customer details, and enforce basic rules such as valid account types, valid customer links, and non-negative balances. It will also allow multiple users and systems to work with the same data safely and provide better recovery if the system fails.

However, moving to a database will not automatically make the information correct. Incorrect data entered by staff, outdated customer information, poor business processes, or an incorrectly designed database can still produce wrong results. A database also does not decide whether a business decision is appropriate; those rules and checks still need to be defined by the bank.

As a result, the bank must establish clear ownership of data, define who can create or change each type of information, validate information before it is entered, and ensure that updates are made in the central system rather than in separate spreadsheets. Staff and applications must also follow the agreed procedures, with appropriate access controls, backups, and regular checks of data quality.

